Aim : Smart Water Tank Monitoring & Automatic Pump Control.

Code :

```
#include <WiFi.h>
#include <AdafruitIO_WiFi.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// ---------- ADAFRUIT IO & WIFI ----------
#define IO_USERNAME  "Nihal_7"
#define IO_KEY       "aio_gfxW34GBrLLB4PSNujVlu56N3RZY"
#define WIFI_SSID    "code"
#define WIFI_PASS    "code1234"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

// ---------- ADAFRUIT IO FEEDS ----------
AdafruitIO_Feed *waterLevelFeed = io.feed("water-level");
AdafruitIO_Feed *distanceFeed = io.feed("distance");
AdafruitIO_Feed *pumpFeed = io.feed("pump-status");
AdafruitIO_Feed *buzzerFeed = io.feed("buzzer-status");
AdafruitIO_Feed *flowFeed = io.feed("flow-rate");
AdafruitIO_Feed *consumptionFeed = io.feed("water-consumed");
AdafruitIO_Feed *temperatureFeed = io.feed("temperature");
AdafruitIO_Feed *tdsFeed = io.feed("tds");

// ---------- PIN CONFIGURATION ----------
#define TRIG_PIN    2
#define ECHO_PIN    4
#define RELAY_PIN   21
```

```
#define BUZZER_PIN  23
#define FLOW_PIN    22
#define TEMP_PIN    19
#define TDS_PIN     18  // Use an ADC-capable ESP32 pin

// ---------- TANK CALIBRATION ----------
const float EMPTY_DISTANCE_CM = 22.0;
const float FULL_DISTANCE_CM  = 4.0;

// ---------- SENSOR SETTINGS ----------
const float FLOW_PULSES_PER_LITRE = 450.0;
const float TDS_FACTOR = 0.5;

const bool RELAY_ACTIVE_LOW = true;
const bool BUZZER_ACTIVE_HIGH = true;

// ---------- TEMPERATURE SENSOR ----------
OneWire oneWire(TEMP_PIN);
DallasTemperature tempSensor(&oneWire);

// ---------- FLOW SENSOR ----------
volatile unsigned long flowPulses = 0;

float flowRate = 0.0;
float totalLitres = 0.0;

unsigned long previousFlowMillis = 0;
unsigned long lastReadMillis = 0;
unsigned long lastPublishMillis = 0;
```

```
// ---------- PUMP STATE ----------
bool pumpOn = false;

// ---------- ULTRASONIC ----------
float readDistanceCM() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(3);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration == 0) {
    return -1.0;
  }

  return duration * 0.0343 / 2.0;
}

// ---------- RELAY CONTROL ----------
void setPump(bool on) {
  pumpOn = on;

  if (RELAY_ACTIVE_LOW) {
    digitalWrite(RELAY_PIN, on ? LOW : HIGH);
  } else {
    digitalWrite(RELAY_PIN, on ? HIGH : LOW);
  }
```

```
}

// ---------- BUZZER CONTROL ----------
void setBuzzer(bool on) {
  if (BUZZER_ACTIVE_HIGH) {
    digitalWrite(BUZZER_PIN, on ? HIGH : LOW);
  } else {
    digitalWrite(BUZZER_PIN, on ? LOW : HIGH);
  }
}

// ---------- FLOW INTERRUPT ----------
void IRAM_ATTR flowISR() {
  flowPulses++;
}

void setup() {
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(FLOW_PIN, INPUT_PULLUP);
  pinMode(TDS_PIN, INPUT);

  // Keep pump OFF during startup
  if (RELAY_ACTIVE_LOW) {
    digitalWrite(RELAY_PIN, HIGH);
  } else {
```

```
    digitalWrite(RELAY_PIN, LOW);
  }

  setBuzzer(false);
  pumpOn = false;

  attachInterrupt(
    digitalPinToInterrupt(FLOW_PIN),
    flowISR,
    RISING
  );

  tempSensor.begin();

  analogReadResolution(12);

  previousFlowMillis = millis();

  // Start cloud connection without waiting indefinitely
  io.connect();

  Serial.println("=====================================");
  Serial.println(" SMART WATER TANK MONITORING SYSTEM");
  Serial.println("       ADAFRUIT IO CLOUD");
  Serial.println("=====================================");
  Serial.println("Pump is OFF at startup.");
  Serial.println("Connecting to Adafruit IO...");
}

void loop() {
```

```
// Maintain Wi-Fi and cloud connection
io.run();

unsigned long now = millis();

// Read sensors every 2 seconds
if (now - lastReadMillis < 2000) {
  return;
}

lastReadMillis = now;

// ---------- WATER LEVEL ----------
float distance = readDistanceCM();
float waterLevel = -1.0;

if (distance >= 0) {
  waterLevel =
    (EMPTY_DISTANCE_CM - distance) /
    (EMPTY_DISTANCE_CM - FULL_DISTANCE_CM) * 100.0;

  waterLevel = constrain(waterLevel, 0.0, 100.0);
}

// ---------- PUMP CONTROL ----------
// Retain previous pump state if sensor reading fails
if (waterLevel >= 0) {
  if (waterLevel < 90.0) {
    setPump(true);
  } else {
```

```
    setPump(false);
  }
}

// ---------- BUZZER ----------
bool alarm = false;

if (waterLevel >= 0) {
  alarm = (waterLevel < 20.0 ||
           waterLevel >= 90.0);
}

setBuzzer(alarm);

// ---------- FLOW RATE & CONSUMPTION ----------
unsigned long currentMillis = millis();
unsigned long elapsed = currentMillis - previousFlowMillis;

if (elapsed >= 2000) {
  noInterrupts();
  unsigned long pulses = flowPulses;
  flowPulses = 0;
  interrupts();

  float litres = pulses / FLOW_PULSES_PER_LITRE;

  float measuredFlow = litres * 60000.0 / elapsed;

  // Display/report zero below 4 L/min
  flowRate = (measuredFlow < 4.0) ? 0.0 : measuredFlow;
```

```
  // Accumulate actual measured consumption
  totalLitres += litres;

  previousFlowMillis = currentMillis;
}

// ---------- WATER TEMPERATURE ----------
tempSensor.requestTemperatures();
float temperature = tempSensor.getTempCByIndex(0);

bool temperatureValid =
  temperature != DEVICE_DISCONNECTED_C &&
  temperature > -100.0;

// ---------- TDS MEASUREMENT ----------
unsigned long adcSum = 0;

for (int i = 0; i < 10; i++) {
  adcSum += analogRead(TDS_PIN);
  delay(5);
}

float adcAverage = adcSum / 10.0;
float voltage = adcAverage * 3.3 / 4095.0;

float compensation = 1.0;

if (temperatureValid &&
    temperature > 0.0 &&
```

```
    temperature < 100.0) {
  compensation =
    1.0 + 0.02 * (temperature - 25.0);
}

float compensatedVoltage = voltage / compensation;

float tdsPPM =
  (133.42 * compensatedVoltage *
     compensatedVoltage * compensatedVoltage
   - 255.86 * compensatedVoltage *
     compensatedVoltage
   + 857.39 * compensatedVoltage)
     * TDS_FACTOR;

if (tdsPPM < 0) {
  tdsPPM = 0;
}

// ---------- SERIAL MONITOR ----------
Serial.println("\n-------- WATER TANK STATUS --------");

if (distance >= 0) {
  Serial.print("Distance       : ");
  Serial.print(distance, 1);
  Serial.println(" cm");

  Serial.print("Water Level    : ");
  Serial.print(waterLevel, 1);
  Serial.println(" %");
```

```
} else {
  Serial.println("Water Level    : SENSOR ERROR");
}

Serial.print("Pump Status    : ");
Serial.println(pumpOn ? "ON" : "OFF");

Serial.print("Buzzer Status  : ");
Serial.println(alarm ? "ON (ALARM)" : "OFF");

Serial.print("Flow Rate      : ");
Serial.print(flowRate, 2);
Serial.println(" L/min");

Serial.print("Water Consumed : ");
Serial.print(totalLitres, 2);
Serial.println(" L");

Serial.print("Temperature    : ");
if (temperatureValid) {
  Serial.print(temperature, 2);
  Serial.println(" C");
} else {
  Serial.println("SENSOR ERROR");
}

Serial.print("TDS            : ");
Serial.print(tdsPPM, 0);
Serial.println(" ppm");
```

```
// ---------- ADAFRUIT IO PUBLISH ----------
// Publish 8 feeds every 20 seconds
if (millis() - lastPublishMillis >= 20000) {
  lastPublishMillis = millis();

  if (io.status() == AIO_CONNECTED) {
    bool publishOK = true;

    if (waterLevel >= 0) {
      publishOK &= waterLevelFeed->save(waterLevel);
      publishOK &= distanceFeed->save(distance);
    }

    publishOK &= pumpFeed->save(pumpOn ? 1 : 0);
    publishOK &= buzzerFeed->save(alarm ? 1 : 0);
    publishOK &= flowFeed->save(flowRate);
    publishOK &= consumptionFeed->save(totalLitres);

    if (temperatureValid) {
      publishOK &= temperatureFeed->save(temperature);
    }

    publishOK &= tdsFeed->save(tdsPPM);

    if (publishOK) {
      Serial.println("Adafruit IO: Data queued.");
    } else {
      Serial.println("Adafruit IO: Publish error.");
    }
  } else {
```

```
      Serial.println("Adafruit IO: Not connected.");
    }
  }

  Serial.println("-----------------------------------");
}
```